Міністерство освіти і науки України
Відокремлений структурний підрозділ «Фаховий коледж електронних приладів
Івано-Франківського національного технічного університету нафти і газу»
Циклова комісія «Інформаційних технологій»

КУРСОВИЙ ПРОЄКТ

з дисципліни «Об’єктно-орієнтоване програмування»

на тему: «Розробка програмного додатку для роботи з базою даних предметної області «Принтери»

ПОЯСНЮВАЛЬНА ЗАПИСКА

КП.ІС-23-01.13.01.00.000 ПЗ

Студента ІІІ курсу ІС-23-01 групи
напряму підготовки 12
спеціальності 126
М.В. Шкварок
Керівник курсового проєкту
викладач О.Я.Стражник

Національна шкала _____
Кількість балів: _____Оцінка: ECTS _____

Члени комісії: _____
(підпис) (прізвище та ініціали)
_____
(підпис) (прізвище та ініціали)
_____
(підпис) (прізвище та ініціали)

Дата захисту «___» _____ 2025 року

Івано-Франківськ – 2025

Відокремлений структурний підрозділ
Фаховий коледж електронних приладів Івано-Франківського національного технічного університету нафти і газу

Дисципліна «Об'єктно – орієнтоване програмування»
Спеціальність «Інформаційні системи та технології»
Курс III Група ІС-23-01 Семестр V

ЗАВДАННЯ НА КУРСОВИЙ ПРОЄКТ

Студент Шкварок Максим Васильович
(прізвище, ім'я, по-батькові)

1 Тема курсового проєкту «Розроблення програмного додатку для роботи з базою даних предметної області «Принтери»

2 Термін здачі студентом закінченої роботи «30»листопада 2025 р.

3 Вихідні дані:обсяг даних з предметної області принтери.

4 Зміст пояснювальної записки (перелік питань, що їх належить розробити)
Вступ
1 Постановка завдання:
- 1.1 Уточнення постановки завдання
- 1.2 Аналіз предметної області та вибір програмних засобів для вирішення поставленого завдання.

2 Аналіз предметної області та виділення об'єктів предметної області.
- 2.1 Опис та розроблення класів
- 2.2 Опис та розроблення методів доступу до даних

3 Проєктування програмного продукту.
- 3.1 Перелік та призначення режимів, структура діалогу

4 Розробка програмного продукту.
5 Відладка програм:
- 5.1 Приклад виконання програмного додатку з тестовими даними та отримання результатів

Висновки
Список використаних джерел
Додаток А Код програми

5 Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

6 Дата видачі завдання " 17 " вересня 2025 р.

КАЛЕНДАРНИЙ ПЛАН

Номер і назва етапів курсового проєкту	**Термін виконання етапів проєкту**	**Примітка**
Аналіз предметної області для створення сутностей бази даних та зв’язків між ними	25.09.2025	виконано
Розроблення бази даних	9.10.2025	виконано
Проєктування класів сутностей	23.10.2025	виконано
Розроблення графічного інтерфейсу для реалізації керування базою даних і запрограмовування взаємодії з базою даних.	6.11.2025	виконано
Відладка програмного додатку	20.11.2025	виконано
Оформлення пояснювальної записки	26.11.2025	виконано
Підготовка проєкту до захисту	27.11.2025	виконано
Захист курсового проєкту		виконано

Студент _____ Шкварок М.В.
(підпис) (прізвище та ініціали)

Керівник проєкту_____ Стражник О.Я.
(підпис) (прізвище та ініціали)

АНОТАЦІЯ

Курсовий проєкт виконаний на: 37 сторінках формату А4 з 11 рисунками, 1 додатком та 4 посиланнями на використані джерела.

Тема роботи: Розробка програмного додатку для роботи з базою даних предметної області «Принтери».

Курсовий проєкт передбачає створення програмного забезпечення для автоматизації процесу обліку, керування та перегляду інформації про принтери, МФУ, їх характеристики, відгуки користувачів, зображення та супутню інформацію (FAQ, новини). Основна мета – розробити зручний, надійний та функціональний інструмент для зберігання, редагування, перегляду та фільтрації даних у межах предметної області.

У роботі проведено аналіз предметної області, визначено основні сутності та їх взаємозв’язки, сформульовано вимоги до програмного забезпечення. Виконано проєктування бази даних у форматі SQLite3, створено графічний інтерфейс користувача з використанням PyQt5, реалізовано основні функції CRUD-операцій (створення, читання, оновлення, видалення) для всіх сутностей.

Програмний додаток дозволяє ефективно керувати даними про пристрої, переглядати їх характеристики та фото, керувати відгуками, додавати FAQ та новини, а також виконувати пошук і фільтрацію за різними параметрами. Інтуїтивно зрозумілий інтерфейс забезпечує зручне використання системи адміністраторами або користувачами.

В результаті виконання роботи створено повнофункціональний застосунок, який відповідає поставленим вимогам, пройшов тестування та демонструє стабільну роботу при виконанні основних операцій із базою даних.

Ключові слова: принтери, МФУ, характеристики, відгуки, зображення, новини, FAQ, база даних, Python, PyQt5, SQLite, інтерфейс.

Відокремлений структурний підрозділ
«Фаховий коледж електронних приладів
Івано-Франківського національного технічного університету нафти і газу»

РЕЦЕНЗІЯ

на курсовий проєкт (роботу)

Студента _____Шкварка Максима Васильовича _____
(прізвище, ім'я, по батькові)
Курсу ____III_____ Групи _____ІС-23-01_____

Спеціальності 126 «Інформаційні системи та технології»
(шифр і назва спеціальності)

Тема курсового проєкту (роботи) «Розробка програмного додатку для роботи з базою даних предметної області «Принтери»

Оцінка якісного рівня курсового проєкту (роботи)

Найменування виду роботи	Оцінка за 12-бальною шкалою
1. Відповідність змісту проєкту (роботи) технічному завданню на проєктування	
2. Новизна розробки та актуальність теми	
3. Ступінь складності розробки	
4. Правильність та повнота виконання розрахунків	
5. Правильність і якість виконання графічної частини	
6. Повнота і якість оформлення пояснювальної записки	
7. Ступінь самостійності виконання проєкту (роботи)	
8. Вміння працювати з науково-технічною літературою	
9. Вміння працювати з комп'ютерною технікою та комп'ютерними програмами	
10. Степінь використання технічної документації, стандартів	
11. Оцінка стилю та грамотності викладення тексту	

Зауваження

_____
_____
_____
_____
_____
_____

Допускається до захисту
Керівник курсового проєкту _____ О.Я. Стражник
(підпис) (прізвище та ініціали)

«___» _____ 2025 року

ЗМІСТ

с.

ВСТУП 6

1 Постановка завдання 7

1.1 Уточнення постановки завдання 7

1.2 Аналіз предметної області та вибір програмних засобів для вирішення поставленого завдання …….. 7

2 Конструкторська частина 9

2.1 Проєктування та створення бази даних та її опис 9

2.2 Побудова ER – діаграми ……..... 11

2.3 Проєктування та розроблення класів програмного додатку ……... 13

2.4 Проєктування та розроблення графічного інтерфейсу програмного забезпечення ……... 16

3 Програмна частина 21

3.1. Опис та розроблення класів 21

3.2 Опис та розроблення методів доступу до даних ……. 24

4 Експериментальна частина …….. 28

4.1 Приклад виконання програмного додатку з тестовими даними та отримання результатів …….. 28

Висновки ……..…..…. 35

Список використаних джерел …….. 36

Додаток А Код програми …….. 38

					КП.ІС-23-01.13.01.00.000 ПЗ		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Шкварок М.В.			Розроблення програмного додатку для роботи з базою даних предметної області «Принтери»	Літ.	Арк. 5 / Аркушів 38
Перевір.		Стражник О.Я.					
Н. контр.						ВСП ФКЕП ІФНТУНГ, ІС-23-01	

ВСТУП

У сучасних умовах розвитку інформаційних технологій зростає потреба в ефективному обліку та управлінні інформацією про техніку, зокрема принтери та багатофункціональні пристрої (МФУ). Традиційне ведення даних у паперових журналах або електронних таблицях часто призводить до помилок, дублювання інформації та ускладнює пошук і редагування записів.

Автоматизація цього процесу дозволяє значно підвищити ефективність обліку, забезпечити швидкий доступ до актуальної інформації та зменшити кількість ручних операцій.

Метою курсового проєкту є розробка програмного додатку для обліку принтерів та МФУ, який надає можливість зберігати, редагувати, переглядати та фільтрувати інформацію про пристрої, їх характеристики, відгуки користувачів, зображення, FAQ та новини.

Основний функціонал програми передбачає:

– додавання, редагування та видалення записів про пристрої та їх характеристики;

– перегляд фото та детальних технічних характеристик пристрою;

– керування відгуками, FAQ та новинами;

– зручну фільтрацію та пошук даних за різними параметрами;

– інтуїтивно зрозумілий графічний інтерфейс користувача.

У ході виконання курсового проєкту набуто загальні і спеціальні компетентності, передбачених освітньо-професійною програмою спеціальності 126 «Інформаційні системи та технології»:

– знання та розуміння предметної області (ЗК5);

– уміння працювати з різними джерелами інформації, аналізувати документацію, бібліотеки й програмні рішення (ЗК6);

– здатність застосовувати набуті знання у практичних умовах при розробці повноцінного програмного продукту (ЗК7).

У межах спеціальних компетентностей було сформовано:

– здатність накопичувати та систематизувати професійні знання, працювати з сучасними інструментами розробки та розуміти необхідність безперервного навчання (СК2);

– застосовувати фундаментальні та міждисциплінарні знання – принципи ООП, моделювання даних, побудову інтерфейсів, алгоритмізацію, роботу з БД – для розв’язання реальних прикладних завдань (СК3).

Отримані результати навчання підтверджують сформованість умінь аналізувати завдання, розробляти алгоритми їх виконання та реалізовувати їх засобами сучасних інформаційних технологій (РН1), систематизувати інформацію щодо методів і технологій розробки програмного забезпечення (РН2), а також застосовувати знання з предметної області на практиці (РН16).

Таким чином, виконання курсового проєкту не лише дозволило створити працездатний програмний продукт, але й забезпечило формування ключових компетентностей, необхідних для подальшого професійного розвитку фахівця з інформаційних технологій.

Для реалізації проєкту використано мову програмування Python та бібліотеку PyQt5 для створення графічного інтерфейсу. Збереження та обробка даних здійснюється за допомогою бази даних SQLite, що забезпечує простоту інтеграції та надійність зберігання інформації.

Розроблений програмний додаток призначений для використання адміністраторами та користувачами з метою зручного управління обліком принтерів та МФУ, підвищення ефективності роботи та зменшення ризику помилок при роботі з даними.

ПОСТАНОВКА ЗАВДАННЯ

1.1 Уточнення постановки завдання

Метою курсового проєкту є розробка програмного додатку для автоматизації обліку принтерів та багатофункціональних пристроїв (МФУ). Система має забезпечити зручний графічний інтерфейс користувача, що дозволяє ефективно керувати інформацією про пристрої, їх технічні характеристики, відгуки користувачів, зображення, FAQ та новини.

Основним завданням програмного забезпечення є створення інструменту, який дає змогу швидко додавати, редагувати та переглядати дані про пристрої, уникаючи дублювання інформації та помилок при ручному введенні.

Програма повинна надавати користувачу можливість виконувати такі дії:

- зчитувати дані з бази та відображати їх у табличному вигляді;
- додавати нові записи про пристрої та їх характеристики;
- редагувати існуючі записи;
- видаляти непотрібні дані;
- переглядати детальні характеристики пристрою та його фото;
- керувати відгуками, FAQ та новинами для кожного пристрою;
- здійснювати пошук та фільтрацію за різними критеріями (наприклад, за назвою пристрою, виробником, типом, ціною).

Для ефективної реалізації функціональних можливостей системи важливо правильно спроєктувати базу даних, визначивши основні сутності (пристрої, виробники, типи пристроїв, характеристики, відгуки, зображення, FAQ, новини), їх атрибути та взаємозв’язки між ними..

1.2 Аналіз предметної області та вибір програмних засобів для вирішення поставленого завдання

У сучасних офісах, навчальних та виробничих закладах велике значення має ефективний облік і контроль комп’ютерної техніки, зокрема принтерів та багатофункціональних пристроїв (МФУ). Відсутність централізованого обліку може

призводити до дублювання записів, помилок у характеристиках пристроїв, втрати інформації про відгуки користувачів, наявність зображень та FAQ, що ускладнює управління парком обладнання.

Для розв'язання цієї проблеми доцільно створити програмний додаток, який забезпечить централізоване зберігання та автоматизовану обробку даних про пристрої. Це дозволить зменшити кількість помилок, спростити оновлення інформації, забезпечити зручний перегляд характеристик та фото пристроїв, а також керування відгуками, FAQ та новинами, пов'язаними з обладнанням.

Мова програмування Python має широкий набір бібліотек для роботи з базами даних та графічними інтерфейсами, а також завдяки активній спільноті, яка забезпечує доступ до численних прикладів та рішень. Python дозволяє швидко реалізувати CRUD-функції (створення, читання, оновлення та видалення даних), що є ключовими для додатку обліку пристроїв [1].

Бібліотека PyQt5 використана для створення графічного інтерфейсу. Вона дозволяє реалізувати сучасні вікна з таблицями, формами та кнопками, забезпечує інтерактивність і підтримує різноманітні стилі оформлення. PyQt5 також добре інтегрується з Python та дозволяє швидко налаштувати відображення складних даних, таких як характеристики пристроїв, фотографії та відгуки користувачів [2].

СУБД SQLite3 обрана для збереження даних завдяки своїй легкості, відсутності необхідності у серверній частині та стабільності роботи з локальними базами даних. SQLite забезпечує надійне зберігання інформації про пристрої, виробників, типи обладнання, відгуки, фотографії та FAQ. Вона інтегрується з Python без додаткових драйверів, що спрощує розгортання програми на різних комп'ютерах [3].

Visual Studio Code використано як середовище розробки завдяки його підтримці Python, можливості налагодження коду, інтеграції з системами контролю версій та розширенням для роботи з базами даних.

Поєднання даних програмних засобів забезпечує зручний, сучасний та надійний інструмент для розробки програмного додатку обліку та управління принтерами і МФУ.

1 КОНСТРУКТОРСЬКА ЧАСТИНА

2.1 Проєктування та створення бази даних та її опис

Для реалізації програмного додатку було спроєктовано реляційну базу даних, яка забезпечує зберігання та обробку всієї необхідної інформації про пристрої, виробників, характеристики, відгуки, зображення та супутні дані. База даних створена у форматі SQLite3 і містить декілька основних таблиць, що взаємопов’язані між собою через зовнішні ключі.

Спроєктовано такі основні таблиці бази даних та їх призначення.

Таблиця Manufacturers – зберігає інформацію про виробників пристроїв. Містить поля:

– id (integer primary key autoincrement) – унікальний ідентифікатор виробника;

– name (text not null) – назва виробника;

– country (text) – країна-виробник;

– website (text) – офіційний сайт виробника.

Таблиця DeviceTypes – містить типи пристроїв (принтери, МФУ тощо). Поля таблиці:

– id (integer primary key autoincrement) – унікальний ідентифікатор типу;

– name (text not null) – назва типу;

– description (text) – опис типу.

Devices – центральна таблиця, що зберігає інформацію про всі пристрої. Поля таблиці:

– id (integer primary key autoincrement) – унікальний ідентифікатор пристрою;

– name (text not null) – назва пристрою;

– manufacturer_id (integer not null) – посилання на виробника;

– device_type_id (integer not null) – посилання на тип пристрою;

– price (real) – ціна;

– stock_quantity (integer) – кількість на складі;

– description (text) – додатковий опис пристрою.

Specifications – таблиця технічних характеристик пристроїв. Містить поля:

– id (integer primary key autoincrement);

– device_id (integer not null) – зв’язок з пристроєм;

– resolution, print_speed, scan_speed, color, interface, paper_size (text) – основні параметри;

– duplex (boolean) – підтримка двостороннього друку.

Таблиця Reviews – містить відгуки користувачів про пристрої. Має наступні поля:

– id (integer primary key autoincrement);

– device_id (integer not null);

– username (text) – ім’я користувача;

– rating (integer) – оцінка;

– comment (text) – текст відгуку;

– date (text) – дата створення відгуку.

Images – зображення пристроїв:

– id (integer primary key autoincrement);

– device_id (integer not null);

– file_path (text) – шлях до файлу;

– description (text) – опис зображення.

Таблиця FAQs містить найчастіші запитання та відповіді по пристроях. Має поля:

– id (integer primary key autoincrement);

– device_id (integer) – необов’язковий зв’язок з пристроєм (може бути загальне питання);

– question, answer (TEXT) – питання та відповідь.

News – таблиця новин та оновлення. Містить поля:

– id (integer primary key autoincrement);

– title (text) – заголовок новини;

– content (TEXT) – зміст новини;

– date (TEXT) – дата публікації.

Структура бази даних забезпечує логічну організацію інформації та підтримує цілісність даних за рахунок зовнішніх ключів між таблицями:

– кожен пристрій має свого виробника (manufacturer_id) та тип (device_type_id);

– характеристики, відгуки, зображення та FAQ пов’язані з конкретним пристроєм через device_id.

Таке проєктування дозволяє ефективно зберігати та обробляти дані, швидко здійснювати пошук, фільтрацію та сортування, а також реалізовувати зручний графічний інтерфейс для користувача з усіма функціями CRUD..

2.2 Побудова ER – діаграми

ER-діаграма (Entity-Relationship Diagram) відображає логічну структуру бази даних програмного додатку для обліку принтерів та демонструє основні сутності системи й зв’язки між ними. Візуалізація моделі даних дозволяє побачити, як окремі елементи предметної області взаємодіють між собою та формують цілісний механізм роботи з інформацією [3].

До ключових сутностей бази даних належать: виробники (Manufacturers), типи пристроїв (DeviceTypes), пристрої (Devices), характеристики (Specifications), відгуки (Reviews), зображення (Images), поширені запитання (FAQs) та новини (News).

Центральною сутністю системи є таблиця Devices, оскільки саме вона описує основні об’єкти предметної області – принтери та інші пристрої. Саме навколо цієї таблиці формуються зв’язки з іншими елементами бази даних:

– manufacturer_id створює зв’язок між пристроєм і таблицею Manufacturers, визначаючи, яким виробником він був випущений. Це дає змогу групувати пристрої за брендами, а також швидко отримувати додаткову інформацію про виробника;

– device_type_id пов’язує пристрій із таблицею DeviceTypes, що визначає вид обладнання (принтер, МФУ, сканер). Такий підхід дозволяє фільтрувати пристрої за типами та формувати структуру каталогу.

Таблиця Specifications пов’язана з Devices через поле device_id та містить технічні параметри кожного конкретного пристрою: роздільну здатність, швидкість

друку чи сканування, тип інтерфейсу, кольоровість і підтримку дуплексного друку. Цей зв’язок один-до-одного або один-до-багатьох (залежно від моделі пристрою) дає змогу розширювати технічні описи.

Таблиця Reviews також містить поле device_id, що визначає, до якого пристрою належить відгук. Це забезпечує можливість формувати рейтинги та оцінювати популярність пристроїв серед користувачів.

У таблиці Images зберігаються візуальні матеріали, пов’язані з моделлю принтера чи сканера. Зв’язок через поле device_id дозволяє прикріплювати декілька зображень до одного пристрою.

Таблиця FAQs реалізує можливість зберігати типові питання та відповіді, що стосуються конкретних моделей або всієї категорії пристроїв загалом. Поле device_id може бути необов’язковим, що дозволяє створювати як загальні, так і специфічні записи.

Таблиця News є незалежною сутністю та не містить зовнішніх ключів, оскільки зберігає загальні новини або оголошення, пов’язані з тематикою пристроїв, технологій чи оновлень у каталозі.

Таким чином, ER-діаграма наочно демонструє структуру та логіку взаємодії між сутностями системи. Вона відображає, як дані про виробників, типи пристроїв, технічні характеристики, відгуки та зображення об’єднуються навколо центральної таблиці Devices, формуючи цілісну інформаційну модель. Така побудова бази даних забезпечує цілісність, узгодженість та структурованість даних, спрощує розробку, супровід та подальше розширення функціоналу програмного додатку..

ER-діаграма бази даних програмного додатку зображено на рисунку 2.1.

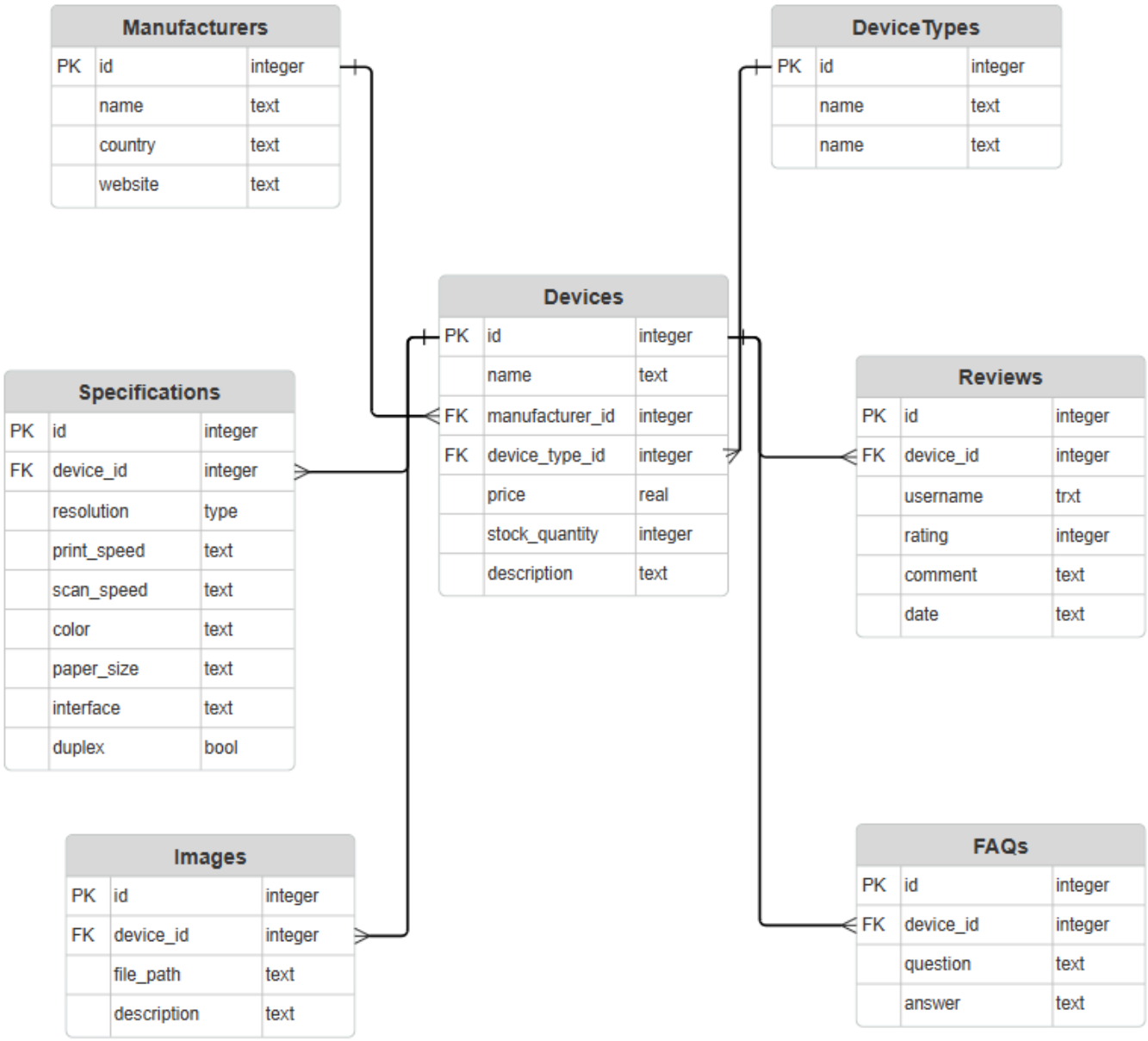


Рисунок 2.1 – ER-діаграма бази даних

2.3 Проєктування та розроблення класів програмного додатку

Для реалізації функціональних можливостей програмного забезпечення, призначеного для обліку, перегляду та аналізу інформації про принтери й інші офісні пристрої, було розроблено низку класів, які відображають основні сутності бази даних. Кожен клас відповідає певній таблиці у сховищі даних і містить необхідні атрибути для зберігання, обробки та відображення інформації у графічному інтерфейсі.

Структура застосунку побудована за принципами об’єктно-орієнтованого програмування, що забезпечує логічність, зрозумілість і модульність системи.

Основними класами програмного додатку є Manufacturer, DeviceType, Device, Specification, Review, Image, FAQ, News.

Клас Manufacturer використовується для опису виробників пристроїв. Він містить базову інформацію, необхідну для ідентифікації компанії та подальшої класифікації техніки. Основні атрибути:

– id (int) – унікальний ідентифікатор виробника;

– name (str) – назва компанії;

– country (str) – країна походження виробника;

– website (str) – офіційний вебсайт.

Клас застосовується під час формування переліку виробників у формі додавання або редагування пристроїв, а також при фільтрації товарів.

Клас DeviceType описує типи офісних пристроїв, що зберігаються в системі: принтери, сканери, БФП тощо. Основні атрибути:

– id (int) – унікальний індекс типу;

– name (str) – назва типу пристрою;

– description (str) – короткий опис або характеристики типу.

Він дозволяє класифікувати обладнання та здійснювати групування для зручного відображення і пошуку.

Клас Device є центральним у системі, оскільки описує основний об’єкт обліку – конкретний принтер або інший офісний пристрій. Основні атрибути:

– id (int) – ідентифікатор пристрою;

– name (str) – назва моделі;

– manufacturer_id (int) – посилання на виробника;

– device_type_id (int) – посилання на тип пристрою;

– price (float) – вартість пристрою;

– stock_quantity (int) – кількість одиниць на складі;

– description (str) – коротка характеристика.

Цей клас використовується для формування каталогу пристроїв, їх пошуку, редагування та перегляду повної інформації.

Клас Specification містить технічні характеристики конкретного пристрою. Він розширює основну сутність Device додатковими параметрами. Основні атрибути:

– id (int) – ідентифікатор запису;

– device_id (int) – зв’язок із пристроєм;

– resolution (str) – роздільна здатність друку чи сканування;

– print_speed (str) – швидкість друку;

– scan_speed (str) – швидкість сканування;

– color (str) – кольоровий чи чорно-білий;

– interface (str) – інтерфейси підключення (USB, Wi-Fi тощо);

– paper_size (str) – підтримувані формати паперу;

– duplex (bool) – наявність автоматичного двостороннього друку.

Цей клас дозволяє відобразити повний технічний профіль кожного пристрою.

Клас Review зберігає інформацію про відгуки користувачів або внутрішні оцінювання пристроїв. Основні атрибути:

– id (int) – ідентифікатор відгуку;

– device_id (int) – посилання на пристрій, якому належить відгук;

– username (str) – ім’я користувача;

– rating (int) – оцінка;

– comment (str) – зміст відгуку;

– date (str) – дата публікації.

Клас використовується для формування блоку відгуків та аналізу якості обладнання.

Клас Image відповідає за зберігання інформації про зображення пристроїв. Його атрибути:

– id (int) – ідентифікатор зображення;

– device_id (int) – посилання на пристрій;

– file_path (str) – шлях до файлу;

– description (str) – короткий опис.

Використовується для відображення галереї пристрою.

Клас FAQ містить часто поставлені питання та відповіді щодо конкретних моделей. Атрибути:

– id (int) – індекс запису;

– device_id (int) – зв’язок із пристроєм;

– question (str) – питання;

– answer (str) – відповідь.

Цей клас підвищує інформативність і зручність користування додатком.

Клас News використовується для створення новинної стрічки, що містить корисні оновлення, оголошення або інформаційні статті. Атрибути:

– id (int) – ідентифікатор новини;

– title (str) – заголовок;

– content (str) – основний текст;

– date (str) – дата публікації.

Розроблена система класів забезпечує повне та логічно впорядковане представлення усіх сутностей, що використовуються у програмному додатку для обліку офісних пристроїв. Кожен клас відповідає окремому аспекту бази даних, а їхня структура дозволяє зберігати необхідні атрибути, ефективно обробляти інформацію та відображати її у користувацькому інтерфейсі. Така модель даних забезпечує масштабованість, зручність подальшого розширення функціональності та полегшує супровід застосунку на всіх етапах його експлуатації.

2.4 Проєктування та розроблення графічного інтерфейсу програмного забезпечення

Графічний інтерфейс користувача (GUI) відіграє ключову роль у взаємодії користувача з програмним забезпеченням, оскільки саме через візуальні елементи здійснюється робота з даними, навігація між функціональними модулями та виконання CRUD-операцій. Для реалізації інтерфейсу програмного додатку «Облік принтерів та багатофункціональних пристроїв» була використана бібліотека PyQt5, яка забезпечує широкий набір сучасних і гнучких віджетів, необхідних для створення зручної та інтуїтивної системи керування даними.

Проєктування інтерфейсу здійснювалось на основі принципів мінімалізму, логічної структуризації та доступності. Інтерфейс поділений на функціональні блоки, що дозволяє користувачу швидко орієнтуватися між різними категоріями пристроїв, характеристиками, виробниками та супутньою інформацією. Основними елементами інтерфейсу є: таблиці, форми введення, кнопки керування, інформаційні панелі, вікна редагування та діалогові повідомлення.

Основні використані віджети PyQt5 програмного додатку:

1. QTableWidget / QTableView – таблиці для відображення даних. Таблиці є центральним елементом інтерфейсу, оскільки саме в табличному вигляді користувач переглядає списки:

– виробників;

– типів пристроїв;

– самих пристроїв;

– характеристик;

– відгуків;

– зображень;

– новин та FAQ.

Таблиці підтримують:

– сортування,

– виділення рядків,

– зміну ширини стовпців,

– автоматичне оновлення після CRUD-операцій.

Це забезпечує зручність перегляду великої кількості даних та швидке отримання потрібної інформації.

2. QPushButton – кнопки керування. У додатку активно застосовуються кнопки, що виконують ключові операції:

– Додати;

– Редагувати;

– Видалити;

– Оновити;

– Завантажити зображення;

– Зберегти;

– Скасувати.

Кнопки мають уніфікований стиль: фіксована ширина, заокруглені кути, єдина колірна схема та відступи між елементами. Це робить інтерфейс візуально цілісним та зрозумілим.

3. QLineEdit – текстові поля введення. Використовуються для введення:

– назв пристроїв;

– імен виробників;

– технічних параметрів;

– назви типу пристрою;

– URL-посилань;

– тексту коротких описів.

Поля мають мінімалістичний дизайн, чіткі межі та внутрішні відступи, що покращує комфорт під час роботи.

4. QTextEdit – багаторядкове текстове поле. Застосовується там, де користувач вводить розширений текст:

– опис пристрою;

– коментар до відгуку;

– зміст FAQ-відповіді;

– текст новини.

Цей віджет дозволяє працювати з великими обсягами інформації, підтримує перенесення рядків та коректне форматування.

5. QComboBox – списки вибору. Важливий віджет, який дозволяє користувачу обрати:

– виробника з існуючого списку;

– тип пристрою;

– колір друку;

– інтерфейс підключення;

– формат паперу.

Комбобокси виключають можливість помилкового введення та забезпечують логічну цілісність даних.

6. QSpinBox / QDoubleSpinBox – числові поля. Використані для введення числових значень:

– ціни;

– кількості на складі;

– рейтингу у відгуках.

Такі поля запобігають некоректному введенню тексту і дозволяють працювати тільки з числами.

7. QLabel – текстові поля та мітки. Мітки використано для:

– заголовків полів вводу,

– підписів до таблиць,

– коротких пояснень,

– виведення статусної інформації.

QLabel допомагає структурувати інтерфейс і робить його зрозумілим навіть для нових користувачів.

8. QHBoxLayout та QVBoxLayout – менеджери компоновки. Застосовані для побудови логічних блоків інтерфейсу:

– розташування кнопок у ряд;

– формування вертикальних форм введення;

– створення вкладених блоків.

Грамотне використання менеджерів компоновки забезпечило адаптивність інтерфейсу та коректне відображення на різних розмірах вікна.

9. QMessageBox – діалогові повідомлення. Використовуються для:

– підтвердження видалення записів;

– сповіщення про помилки;

– повідомлень про успішне додавання або оновлення даних.

Це дозволяє користувачу чітко розуміти результат виконаних дій.

10. QFileDialog – вибір файлів. Важливий для модуля роботи із зображеннями. Користувач може:

– вибрати файл фотографії пристрою;

– зберегти або завантажити дані.

Проєктування графічного інтерфейсу було виконано з урахуванням принципів зручності, доступності та функціональності. Використання бібліотеки PyQt5 дозволило створити сучасну та інтуїтивно зрозумілу структуру, що ефективно поєднує таблиці, форми, кнопки, діалогові вікна та інші елементи. Завдяки продуманому розташуванню віджетів інтерфейс забезпечує швидку навігацію, зручне управління даними та комфортну взаємодію користувача з програмним продуктом..

2 ПРОГРАМНА ЧАСТИНА

3.1 Опис та розроблення класів

Для забезпечення зручної взаємодії між програмним застосунком та базою даних було спроєктовано набір класів, кожен із яких представляє окрему сутність предметної області: виробників пристроїв, типи обладнання, конкретні моделі, технічні характеристики, відгуки користувачів, зображення товарів, FAQ та новини.

Об’єктно-орієнтований підхід дозволив реалізувати логічну модель даних, де кожен запис таблиці бази даних відображається у вигляді окремого об’єкта. Така структура підвищує гнучкість, читаємість і масштабованість коду, а також спрощує подальшу розробку та супровід програмного додатку.

Клас Manufacturer використовується для моделювання виробників пристроїв. Він містить основні атрибути, необхідні для зберігання загальної інформації про компанію-виробника: назву, країну походження та офіційний вебсайт.

Модель застосовується при формуванні переліку виробників та прив’язці їх до конкретних пристроїв. Програмний код класу Manufacturer:

```
class Manufacturer:
    """Модель виробника пристрою."""
    def __init__(self, id=None, name="", country=None, website=None):
        self.id = id
        self.name = name
        self.country = country
        self.website = website

    def __str__(self):
        return f"{self.name} ({self.country})"
```

Клас DeviceType моделює типи офісних пристроїв – наприклад, принтери, сканери, копіювальні апарати чи багатофункціональні пристрої.

Цей клас дозволяє групувати обладнання за категоріями, що спрощує навігацію та фільтрацію у програмі. Код класу:

```
class DeviceType:
    """Модель типу пристрою."""
    def __init__(self, id=None, name="", description=None):
        self.id = id
        self.name = name
        self.description = description

    def __str__(self):
        return self.name
```

Клас Device є центральним у моделі даних та описує конкретний пристрій. Він включає такі атрибути, як назва, виробник, тип пристрою, ціна, кількість на складі та

загальний опис. Об’єкти цього класу відображають основну інформацію про обладнання та використовуються в усіх модулях програми.

Код класу:

```
class Device:
    """Модель пристрою."""
    def __init__(self, id=None, name="", manufacturer_id=None,
device_type_id=None,
                 price=None, stock_quantity=None, description=None):
        self.id = id
        self.name = name
        self.manufacturer_id = manufacturer_id
        self.device_type_id = device_type_id
        self.price = price
        self.stock_quantity = stock_quantity
        self.description = description

    def __str__(self):
        return f"{self.name}"
```

Клас Specification описує технічні характеристики пристрою: роздільну здатність, швидкість друку та сканування, наявність кольорового режиму, інтерфейси підключення, підтримувані формати паперу та можливість двостороннього друку.

Такий клас дозволяє деталізувати інформацію про обладнання та створити окремий модуль технічних параметрів. Код класу:

```
class Specification:
    """Модель технічних характеристик пристрою."""
    def __init__(self, id=None, device_id=None, resolution=None,
print_speed=None,
                 scan_speed=None, color=None, interface=None, paper_size=None,
duplex=False):
        self.id = id
        self.device_id = device_id
        self.resolution = resolution
        self.print_speed = print_speed
        self.scan_speed = scan_speed
        self.color = color
        self.interface = interface
        self.paper_size = paper_size
        self.duplex = duplex

    def __str__(self):
        return f"Характеристики пристрою {self.device_id}"
```

Клас Review використовується для зберігання відгуків користувачів щодо конкретного пристрою. Він містить ім’я автора, оцінку, коментар та дату публікації.

Ця модель дозволяє формувати систему рейтингу та аналізувати досвід користувачів. Код класу:

```
class Review:
    """Модель відгуку користувача."""
    def __init__(self, id=None, device_id=None, username=None, rating=None,
                 comment=None, date=None):
        self.id = id
```

```
            self.device_id = device_id
            self.username = username
            self.rating = rating
            self.comment = comment
            self.date = date

        def __str__(self):
            return f"{self.username}: {self.rating}/5"
```

Клас Image призначений для роботи із зображеннями пристроїв. Він зберігає шлях до файлу зображення та його короткий опис.

Це дозволяє відображати фотографії обладнання в інтерфейсі програми.

Код класу:

```
class Image:
    """Модель зображення пристрою."""
    def __init__(self, id=None, device_id=None, file_path=None,
description=None):
            self.id = id
            self.device_id = device_id
            self.file_path = file_path
            self.description = description

        def __str__(self):
            return f"Зображення для пристрою {self.device_id}"
```

Клас FAQ моделює запитання та відповіді, пов’язані з використанням пристроїв. Він застосовується для формування довідкового розділу. Код класу:

```
class FAQ:
    """Модель FAQ-запитань."""
    def __init__(self, id=None, device_id=None, question=None, answer=None):
        self.id = id
        self.device_id = device_id
        self.question = question
        self.answer = answer

    def __str__(self):
        return f"Питання: {self.question}"
```

Клас News використовується для публікації новин і оголошень. Він містить заголовок, текст новини та дату створення. Код класу:

```
class News:
    """Модель новин."""
    def __init__(self, id=None, title=None, content=None, date=None):
        self.id = id
        self.title = title
        self.content = content
        self.date = date

    def __str__(self):
        return self.title
```

У ході проєктування було створено набір класів, які відображають основні сутності предметної області та забезпечують об’єктно-орієнтоване подання даних у програмному додатку. Кожен клас відповідає окремій таблиці бази даних і містить необхідні атрибути для зберігання, опрацювання та відображення інформації. Такий

підхід забезпечує логічну структуру коду, підвищує його зрозумілість, полегшує взаємодію з базою даних та спрощує реалізацію функціональних можливостей системи. Розроблена модель класів є гнучкою та масштабованою основою для подальшого розширення та модернізації програмного забезпечення.

3.2 Опис та розроблення методів доступу до даних

Для забезпечення взаємодії між програмним застосунком та базою даних printers.db було створено окремий клас DatabaseManager, який виконує всі операції доступу до даних. Клас реалізує CRUD-функціонал (Create, Read, Update, Delete) для всіх сутностей системи: виробники, типи пристроїв, пристрої, характеристики, відгуки, зображення, FAQ та новини.

Централізація доступу до БД у межах одного класу зменшує дублювання коду, полегшує супровід, забезпечує цілісність даних і спрощує взаємодію з іншими частинами програми, зокрема з графічним інтерфейсом.

Код класу DatabaseManager:

```
# database.py

import sqlite3
from models import Manufacturer, DeviceType, Device, Specification, Review, Image, FAQ, News

class DatabaseManager:
    def __init__(self, db_name="printers.db"):
        self.conn = sqlite3.connect(db_name)
        self.conn.row_factory = sqlite3.Row
        self.cursor = self.conn.cursor()
```

Клас відкриває підключення до БД, використовує sqlite3.Row, що дозволяє звертатися до стовпців таблиці за назвами, а не лише за індексами.

Основні методи доступу до даних:

1. Зчитування даних (SELECT). Методи отримання даних необхідні для відображення інформації у таблицях інтерфейсу, комбо-боксах та картках пристроїв.

Кожен метод виконує SQL-запит і перетворює рядки таблиці на об’єкти відповідних моделей.

– як приклад – отримання виробників:

```
def get_manufacturers(self):
    self.cursor.execute("SELECT * FROM Manufacturers")
    return [Manufacturer(**row) for row in map(dict, self.cursor.fetchall())]
```

– отримання всіх пристроїв:

```
def get_devices(self):
    self.cursor.execute("SELECT * FROM Devices")
    return [Device(**row) for row in map(dict, self.cursor.fetchall())]
```

– отримання характеристик конкретного принтера:

```
def get_specification_by_device(self, device_id):
    self.cursor.execute("SELECT * FROM Specifications WHERE device_id = ?",
(device_id,))
    row = self.cursor.fetchone()
    if row:
        return Specification(**dict(row))
    return None
```

Методи читання використовуються для формування списків, таблиць і карток товарів у GUI.

2. Додавання нових записів (INSERT), тобто методи створення нових записів дозволяють поповнювати базу даних:

– додавання виробника:

```
def add_manufacturer(self, manufacturer: Manufacturer):
    self.cursor.execute(
        "INSERT INTO Manufacturers (name, country, website) VALUES (?, ?, ?)",
        (manufacturer.name, manufacturer.country, manufacturer.website)
    )
    self.conn.commit()
    manufacturer.id = self.cursor.lastrowid
    return manufacturer.id
```

– додавання характеристик пристрою:

```
def add_specification(self, spec: Specification):
    self.cursor.execute(
        "INSERT INTO Specifications (device_id, resolution, print_speed,
scan_speed, color, interface, paper_size, duplex)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
        (spec.device_id, spec.resolution, spec.print_speed, spec.scan_speed,
spec.color, spec.interface, spec.paper_size, spec.duplex)
    )
    self.conn.commit()
    spec.id = self.cursor.lastrowid
    return spec.id
```

3. Оновлення даних (UPDATE). Оновлення дозволяє користувачу редагувати характеристики, описи та інші параметри пристроїв.

– оновлення типу пристрою:

```
def update_device_type(self, device_type: DeviceType):
    self.cursor.execute(
        "UPDATE DeviceTypes SET name=?, description=? WHERE id=?",
        (device_type.name, device_type.description, device_type.id)
    )
    self.conn.commit()
```

– оновлення інформації про пристрій:

```
def update_device(self, device: Device):
    self.cursor.execute(
        "UPDATE Devices SET name=?, manufacturer_id=?, device_type_id=?, price=?, stock_quantity=?, description=? WHERE id=?",
        (device.name, device.manufacturer_id, device.device_type_id, device.price, device.stock_quantity, device.description, device.id)
    )
    self.conn.commit()
```

4. Видалення записів (DELETE). Методи видалення забезпечують очищення бази від застарілих чи непотрібних даних.

– видалення відгуку:

```
def delete_review(self, review_id):
    self.cursor.execute("DELETE FROM Reviews WHERE id=?", (review_id,))
    self.conn.commit()
```

– видалення зображення:

```
def delete_image(self, image_id):
    self.cursor.execute("DELETE FROM Images WHERE id=?", (image_id,))
    self.conn.commit()
```

5. Додаткові методи:

```
Отримання зображень певного пристрою
def get_images_by_device(self, device_id):
    self.cursor.execute("SELECT * FROM Images WHERE device_id = ?", (device_id,))
    rows = self.cursor.fetchall()
    return [Image(**dict(r)) for r in rows] if rows else []
```

Ці методи важливі для формування карток принтера з фото, характеристиками та відгуками.

6. Закриття з’єднання:

```
def close(self):
    self.conn.close()
```

Метод використовується при завершенні роботи програми для коректного звільнення ресурсів.

Клас DatabaseManager забезпечує повний цикл роботи з базою даних: від отримання інформації до додавання, оновлення та видалення записів. Завдяки параметризованим SQL-запитам робота системи є надійною та захищеною від SQL-ін’єкцій.

Централізоване зберігання всіх операцій у єдиному класі робить архітектуру застосунку логічною, зрозумілою та легкою для розширення. У майбутньому можна без проблем додавати нові сутності або змінювати структуру бази даних без впливу на інші частини системи.

4 ЕКСПЕРИМЕНТАЛЬНА ЧАСТИНА

4.1 Приклад виконання програмного додатку з тестовими даними та отримання результатів

Після запуску програми main.py користувач бачить головне вікно програмного додатку, яке складається з наступних блоків:

– панель керування (кнопки додавання, редагування, видалення);

– блок фільтрів за категоріями пристроїв та виробниками;

– таблиця пристроїв із основними даними та доступом до деталей.

Головне вікно програми зображено на рисунку 4.1.

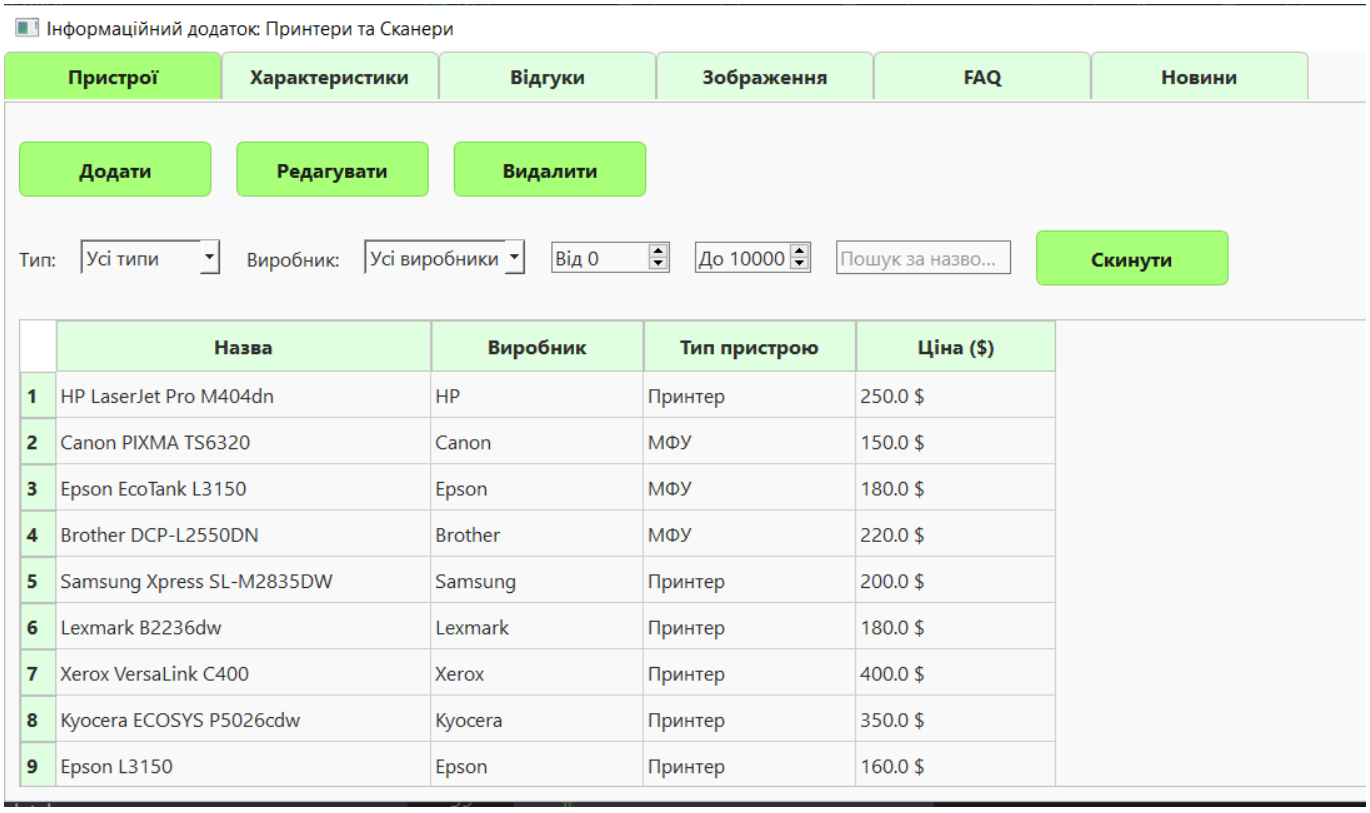


Рисунок 4.1 – Головне вікно програмного додатку «Принтери»

Вкладка «Пристрої» показує список усіх пристроїв із таблицею, де відображаються: назва, виробник, тип пристрою, ціна, наявність на складі.

Основний функціонал вкладки:

– додавання пристрою – кнопка «Додати» відкриває форму з полями: назва, виробник, тип пристрою, ціна, кількість на складі, опис (рисунок 4.2);

– редагування пристрою – змінює дані виділеного пристрою;

– видалення пристрою – видаляє обраний пристрій після підтвердження;

– фільтри та сортування – за типом пристрою, виробником або ціною.

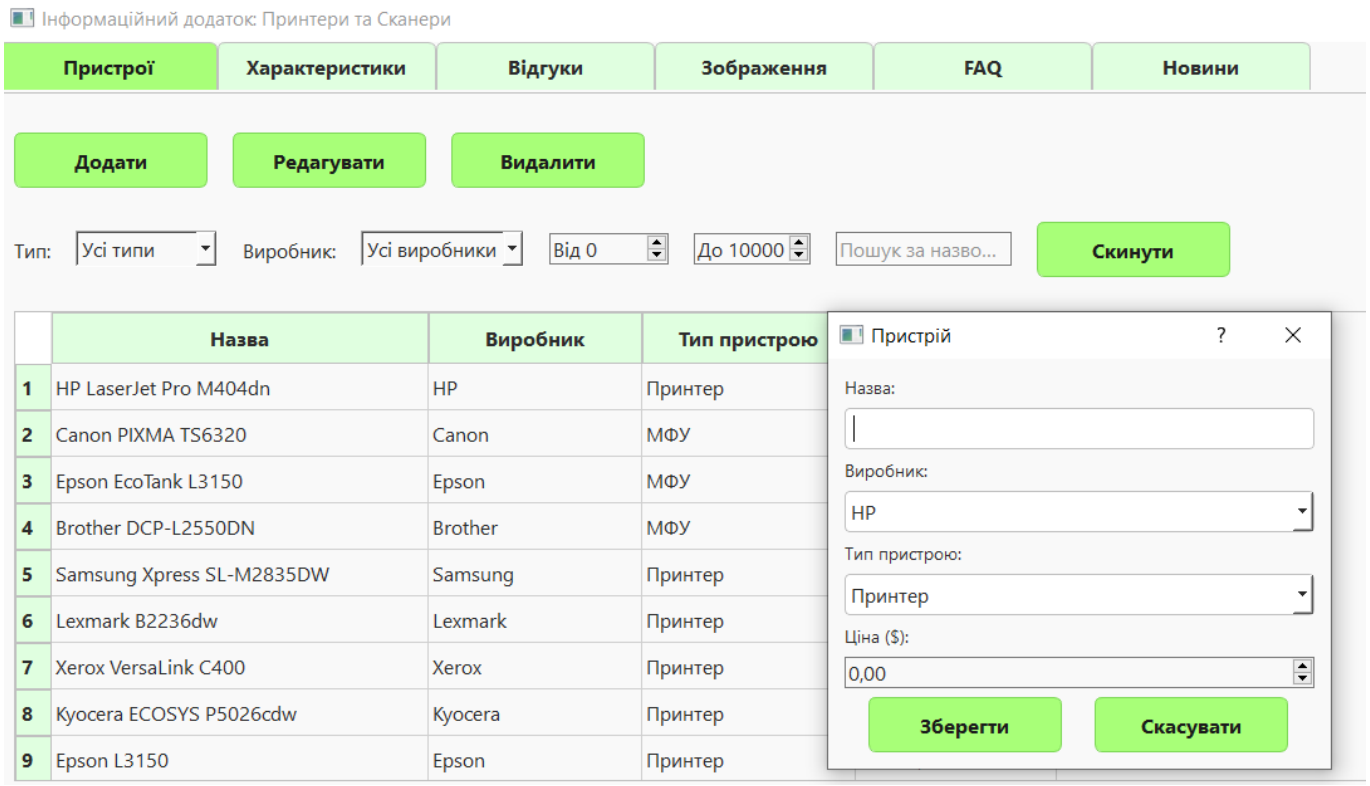


Рисунок 4.2 – Додавання нового пристрою

Деталі принтера показано на рисунку 4.3.

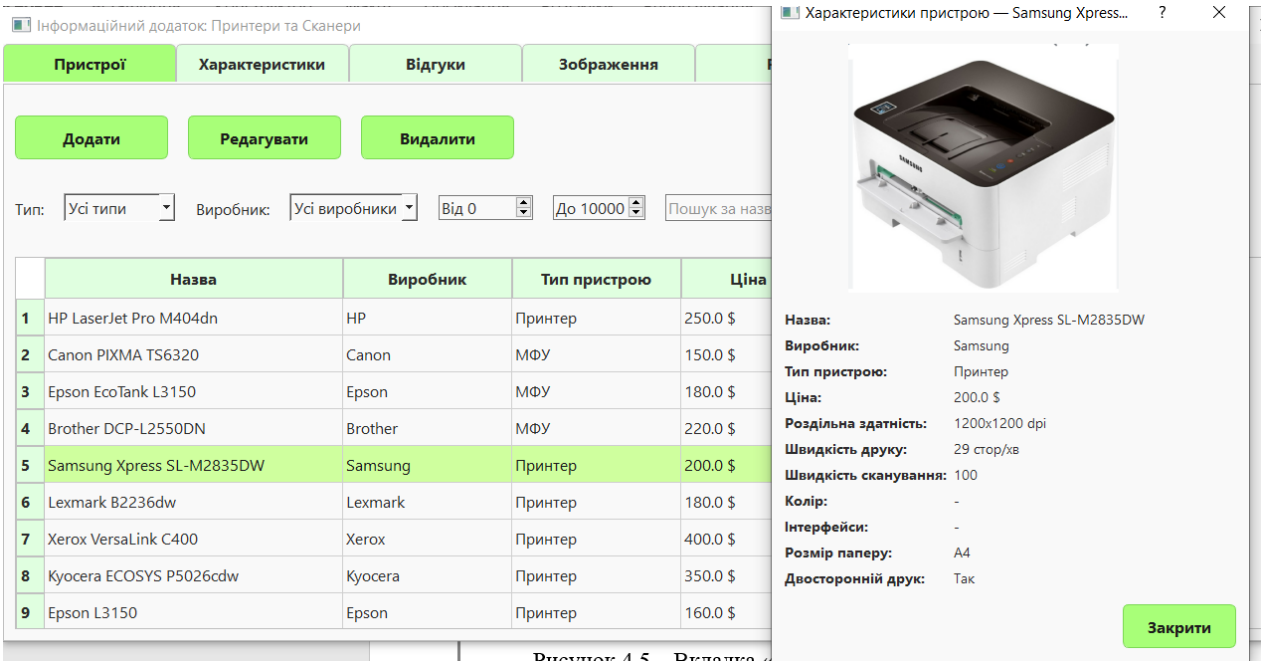


Рисунок 4.3 – Вкладка «Пристрої» з деталями

Результат фільтрування показано на рисунку 4.4.

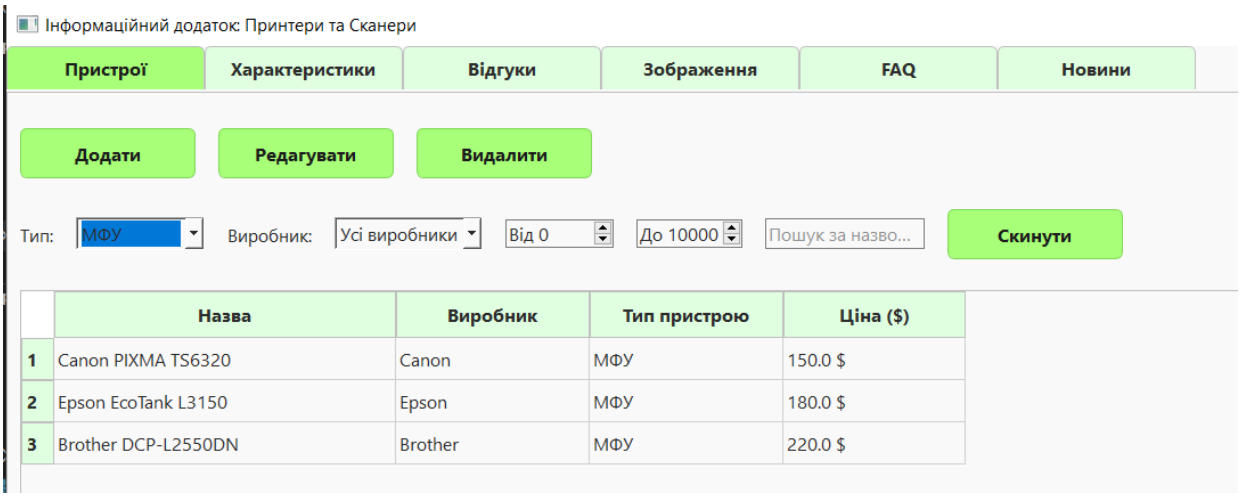


Рисунок 4.4 – Вкладка «Пристрої» з фільтрами

Вкладка «Характеристики», що зображена на рисунку 4.5, дозволяє переглянути та редагувати технічні параметри обраного пристрою:

– роздільна здатність друку;

– швидкість друку/сканування;

– кольоровість;

– підтримувані інтерфейси;

– формат паперу;

– наявність двостороннього друку.

Реалізує функції:

– додавання характеристики – для нового пристрою;

– редагування характеристик – змінює параметри виділеного пристроюд

– видалення характеристики – видаляє технічні дані пристрою.



	Пристрій	Роздільна здатність	Шв. друку	Шв. сканування	Колір	Інтерфейс	Розмір паперу	Двосторонній
1	HP LaserJet Pro M404dn	1200x1200 dpi	38 стор/хв	–			A4	✓
2	Canon PIXMA TS6320	4800x1200 dpi	15 стор/хв	4 стор/хв	Кольоровий,...	USB, Wi-Fi	A4	✓
3	Epson EcoTank L3150	5760x1440 dpi	10 стор/хв	30-35 стор/хв	Кольоровий	USB, Wi-Fi, Bluetooth	A4	✗
4	Brother DCP-L2550DN	1200x1200 dpi	30 стор/хв	22 стор/хв	Кольоровий,...	USB, Wi-Fi, Ethernet	A4	✓
5	Samsung Xpress SL-M2835DW	1200x1200 dpi	29 стор/хв	–			A4	✓
6	Lexmark B2236dw	1200x1200 dpi	26 стор/хв	–	Ч/Б	USB	A4	✗
7	Xerox VersaLink C400	1200x1200 dpi	35 стор/хв	–	Кольоровий	USB/Ethernet	A4	✓
8	Kyocera ECOSYS P5026cdw	1200x1200 dpi	28 стор/хв	–	Кольоровий	USB, Wi-Fi	A4, A5, A3	✓
9	Epson L3150	1200x1200 dpi	10 стор/хв	–	Кольоровий,...	USB, Bluetooth	A4	✗

Рисунок 4.5 – Вкладка «Характеристики»

Додавання характеристики принтера показано на рисунку 4.6.

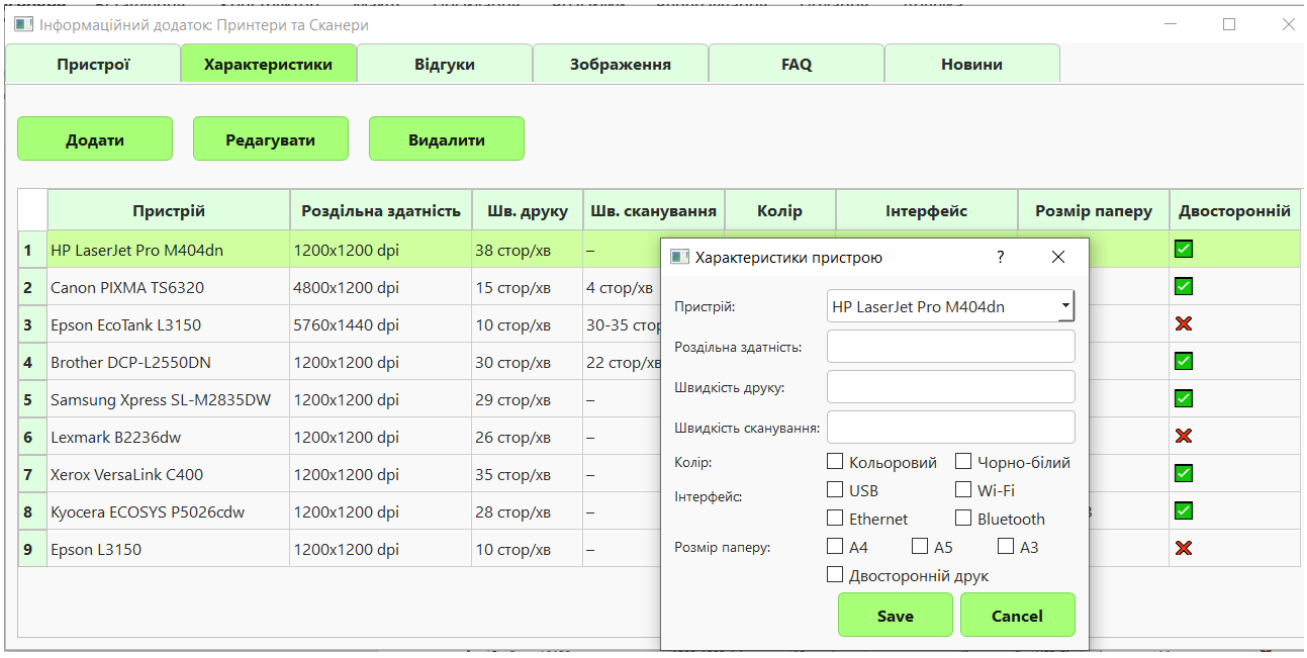


Рисунок 4.6 – Додавання нової характеристики

На вкладці «Відгуки» відображаються коментарі користувачів про пристрої, оцінки та дати додавання.

Основними функціями є:

– Додавання відгуку – ім’я користувача, рейтинг, коментар, дата.

– Редагування відгуку – зміна тексту та рейтингу.

– Видалення відгуку – видалення після підтвердження.

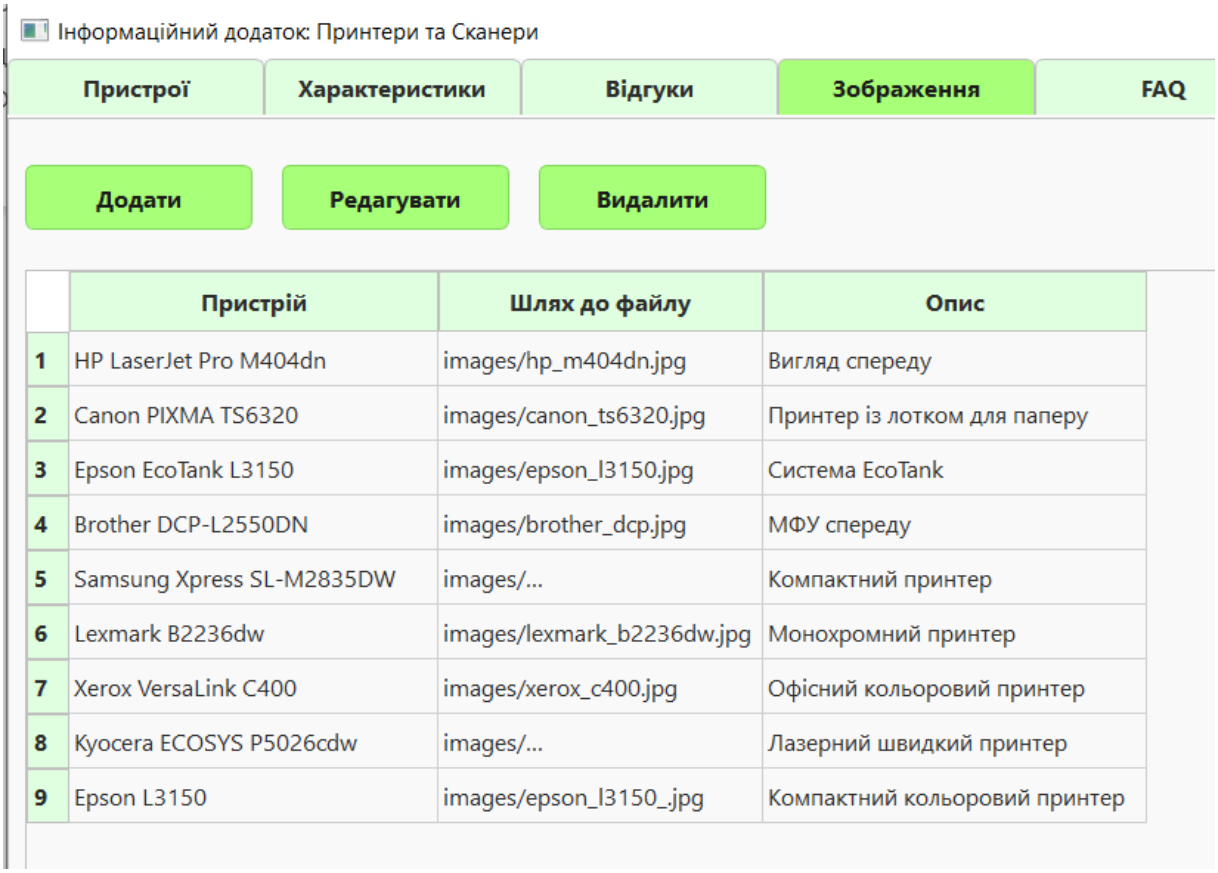
Вкладку «Відгуки» зображено на рисунку 4.7.



	Пристрій	Користувач	Оцінка	Коментар	Дата
1	HP LaserJet Pro M404dn	Ольга	5	Відмінний принтер!	2025-11-10
2	Canon PIXMA TS6320	Андрій	4	Хороша ціна та якість	2025-11-09
3	Epson EcoTank L3150	Софія	5	Обожнюю систему EcoTank	2025-11-08
4	Brother DCP-L2550DN	Іван	4	Надійний принтер	2025-11-07
5	Samsung Xpress SL-M2835DW	Марія	3	Трохи шумний	2025-11-06
6	Lexmark B2236dw	Дмитро	4	Добре працює	2025-11-05
7	Xerox VersaLink C400	Катерина	5	Чудова кольоровий друк	2025-11-04
8	Kyocera ECOSYS P5026cdw	Тарас	4	Хороший офісний принтер	2025-11-03
9	Epson EcoTank L3150	Ольга	5	Безвідмовний принтер	2025-11-10

Рисунок 4.7 – Вкладка «Відгуки» з таблицею оцінок

Вкладка «Зображення» дозволяє додавати фото пристроїв. Кожен запис містить файл зображення та короткий опис, кнопки для додавання, редагування та видалення фото. Дану вкладку показано на рисунку 4.8.



	Пристрій	Шлях до файлу	Опис
1	HP LaserJet Pro M404dn	images/hp_m404dn.jpg	Вигляд спереду
2	Canon PIXMA TS6320	images/canon_ts6320.jpg	Принтер із лотком для паперу
3	Epson EcoTank L3150	images/epson_l3150.jpg	Система EcoTank
4	Brother DCP-L2550DN	images/brother_dcp.jpg	МФУ спереду
5	Samsung Xpress SL-M2835DW	images/...	Компактний принтер
6	Lexmark B2236dw	images/lexmark_b2236dw.jpg	Монохромний принтер
7	Xerox VersaLink C400	images/xerox_c400.jpg	Офісний кольоровий принтер
8	Kyocera ECOSYS P5026cdw	images/...	Лазерний швидкий принтер
9	Epson L3150	images/epson_l3150_.jpg	Компактний кольоровий принтер

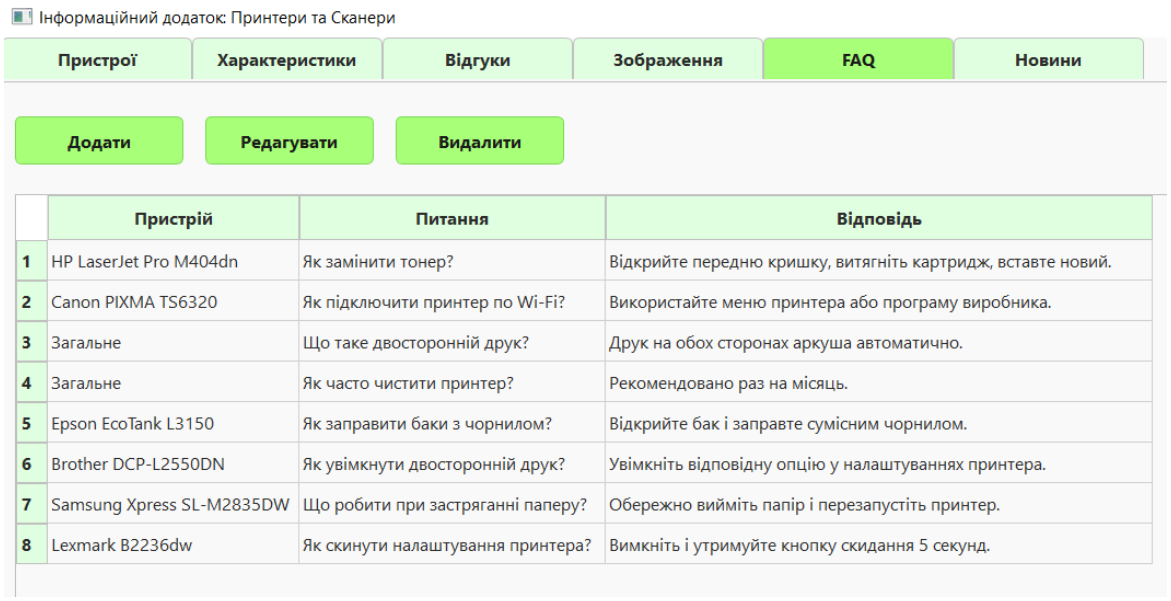
Рисунок 4.8 – Вкладка «Зображення»

Вкладка «FAQ / Поради» показує питання та відповіді щодо конкретних пристроїв. Наявна можливість:

– додавання нового FAQ – питання та відповідь;

– редагування FAQ – зміна існуючих записів;

– видалення FAQ – після підтвердження дії.



	Пристрій	Питання	Відповідь
1	HP LaserJet Pro M404dn	Як замінити тонер?	Відкрийте передню кришку, витягніть картридж, вставте новий.
2	Canon PIXMA TS6320	Як підключити принтер по Wi-Fi?	Використайте меню принтера або програму виробника.
3	Загальне	Що таке двосторонній друк?	Друк на обох сторонах аркуша автоматично.
4	Загальне	Як часто чистити принтер?	Рекомендовано раз на місяць.
5	Epson EcoTank L3150	Як заправити баки з чорнилом?	Відкрийте бак і заправте сумісним чорнилом.
6	Brother DCP-L2550DN	Як увімкнути двосторонній друк?	Увімкніть відповідну опцію у налаштуваннях принтера.
7	Samsung Xpress SL-M2835DW	Що робити при застряганні паперу?	Обережно вийміть папір і перезапустіть принтер.
8	Lexmark B2236dw	Як скинути налаштування принтера?	Вимкніть і утримуйте кнопку скидання 5 секунд.

Рисунок 4.9 – Вкладка «FAQ / Поради»

Вкладка «Новини» відображає останні події та оновлення щодо пристроїв. Можливими функціями є додавання, редагування, видалення новини. Дану вкладку показано на рисунку 4.10.



	Заголовок	Контент	Дата
1	HP випустила новий LaserJet	HP представила нову серію LaserJet M500 для офісів	2025-11-01
2	Canon оновлює PIXMA	Canon оновила серію PIXMA TS з покращеною ...	2025-11-02
3	Epson представляє EcoTank L400	Нове МФУ з більшою ємністю чорнил	2025-11-03
4	Огляд MFP Brother	Серія DCP протестована експертами	2025-11-04
5	Поради для Samsung Xpress	Топ-поради для принтерів Samsung Xpress	2025-11-05
6	Lexmark B серія оновлення	Доступне оновлення прошивки для Lexmark B	2025-11-06
7	Новини Xerox C серії	Xerox VersaLink C400 доступний на нових ринках	2025-11-07
8	Kyocera: обслуговування принтерів	Kyocera ділиться порадами з обслуговування	2025-11-08

Рисунок 4.10 – Вкладка «Новини»

Таким чином, усі вкладки додатку дозволяють повністю керувати інформацією про пристрої, їх виробників, характеристики, відгуки та додаткові дані, забезпечуючи зручний доступ і контроль над базою даних.

ВИСНОВКИ

У процесі виконання курсового проєкту було розроблено функціональний програмний додаток для автоматизації роботи з базою даних предметної області «Принтери».

У під час виконання було реалізовано такі можливості:

– додавання, редагування та видалення записів про принтери з контролем коректності введених даних;

– ведення довідника моделей, включаючи виробника, тип друку та ресурс;

– перегляд списку принтерів у зручній табличній формі з можливістю сортування та пошуку;

– фільтрацію принтерів за моделлю, статусом, локацією або іншими критеріями (залежно від реалізації);

– зручні діалогові вікна для роботи із записами, що забезпечують інтуїтивно зрозумілий інтерфейс.

База даних створена на основі SQLite3, що забезпечує компактність, автономність і простоту інтеграції без додаткових серверних компонентів. Структура БД спроєктована відповідно до вимог реляційного моделювання та охоплює ключові сутності: принтери, моделі, технічний стан та інші пов’язані дані.

Графічний інтерфейс реалізовано за допомогою бібліотеки PyQt5. Завдяки цьому вдалося створити багатовкладкову структуру вікон, таблиці з форматуванням, діалогові форми та продуманий набір керуючих елементів для комфортної роботи користувача.

Результатом роботи став повноцінний застосунок, що дає змогу ефективно організувати облік та контроль за принтерами в установі, скоротити витрати часу на пошук інформації та підвищити якість технічного обслуговування обладнання.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

Основні нормативні акти

– Стандарт фахової передвищої освіти за освітньо-професійним ступенем фаховий молодший бакалавр з галузі знань 12 «Інформаційні технології», спеціальності 126 «Інформаційні системи та технології» затверджений і введений в дію наказом Міністерства освіти і науки України від 17.11.2021 р. № 1246. [Електронний ресурс].– URL: https://mon.gov.ua/static-objects/mon/sites/1/Fakhova%20peredvyshcha%20osvita/Zatverdzheni.standarty/2021/11/18/126-Inform.syst.tekhnol.18.11.pdf

– Освітньо-професійна програма підготовки здобувачів фахової передвищої освіти «Інформаційні системи та технології» ВСП ФКЕП ІФНТУНГу спеціальності 126 «Інформаційні системи та технології» галузі знань 12 «Інформаційні технології». – URL: https://drive.google.com/file/d/1EsdTwcbckM2SIz66Aj0yKtnyHveCNG-5/view

– Положення про систему забезпечення академічної доброчесності у ВСП «Фаховий коледж електронних приладів ІФНТУНГ». – URL: https://kep.nung.edu.ua/about/documents

– Положення про курсові та дипломні роботи у відокремленому структурному підрозділі «Фаховий коледж електронних приладів ІФНТУНГ». – URL: https://kep.nung.edu.ua/about/documents

– Оформлення текстових та графічних документів у проєктах та роботах здобувачів освіти. Методичні рекомендації. / Бігун О.Б., Бідасюк М.А., Білан О.Я., Малик І.В. – Івано-Франківськ, 2023.

– Методичні рекомендації для виконання курсового проєкту з дисципліни «Об’єктно – орієнтованого програмування» для студентів спеціальності 126 «Інформаційні системи та технології». / Укладач: О.Я. Стражник.– 2025, 68с.

1. Васильєв О.М. Програмування в PYTHON. Теорія і практика : навч. посіб. Київ : Видавництво Ліра-К, 2023. 462 с.

2. Офіційна документація по мові програмуванню PyQt5 [Електронний ресурс] – URL: https://python-scripts.com/pyqt5

3. Офіційна документація по SQLite [Електронний ресурс] – URL: https://www.sqlite.org/docs.html.

4. Основні етапи проєктування баз даних [Електронний ресурс] – URL: https://javarush.com/ua/quests/lectures/ua.questhibernate.level17.lecture01

5. Що таке дизайн графічного інтерфейсу користувача GUI [Електронний ресурс] – URL: https://training.qatestlab.com/blog/technical-articles/what-is-graphical-user-interface-design/

Додаток А

Програмний код Python

Код файлу schedule_tab.py

```
from PyQt5.QtWidgets import (
    QWidget,     QVBoxLayout,     QHBoxLayout,     QPushButton,     QTableWidget,
QTableWidgetItem,
    QLabel, QDialog, QFormLayout, QComboBox, QMessageBox
)
from PyQt5.QtCore import Qt
from models.database import get_connection


class ScheduleDialog(QDialog):
    """Діалог для додавання або редагування запису розкладу"""
    WEEK_MAP = {"Чисельник": "upper", "Знаменник": "lower", "Щотижня": "both"}
    REVERSE_WEEK_MAP = {v: k for k, v in WEEK_MAP.items()}

    def __init__(self, parent=None, data=None):
        super().__init__(parent)
        self.setWindowTitle("Розклад")
        self.layout = QFormLayout(self)

        self.day_cb = QComboBox()
        self.lesson_cb = QComboBox()
        self.group_cb = QComboBox()
        self.subject_cb = QComboBox()
        self.teacher_cb = QComboBox()
        self.audience_cb = QComboBox()
        self.week_cb = QComboBox()

        self.load_comboboxes()

        # сигнал під'єднаємо пізніше
        self.group_cb.currentIndexChanged.connect(self.update_subjects_by_group)

        self.layout.addRow("День:", self.day_cb)
        self.layout.addRow("Пара:", self.lesson_cb)
        self.layout.addRow("Група:", self.group_cb)
        self.layout.addRow("Предмет:", self.subject_cb)
        self.layout.addRow("Викладач:", self.teacher_cb)
        self.layout.addRow("Аудиторія:", self.audience_cb)
        self.layout.addRow("Тип тижня:", self.week_cb)

        self.btn_save = QPushButton("Зберегти")
        self.btn_save.clicked.connect(self.accept)
        self.layout.addWidget(self.btn_save)

        # --- якщо редагування ---
        if data:
            self.schedule_id = data[0]

            # ❗ спершу заповнюємо без фільтрації
            self.day_cb.setCurrentIndex(self.day_cb.findData(data[1]))
            self.lesson_cb.setCurrentIndex(self.lesson_cb.findData(data[2]))
            self.group_cb.setCurrentIndex(self.group_cb.findData(data[3]))

            # ✅ якщо предмет знайдено у списку — вибрати
            idx = self.subject_cb.findData(data[4])
            if idx != -1:
                self.subject_cb.setCurrentIndex(idx)
            else:
                # якщо предмет відсутній (через фільтрацію) — додаємо його вручну
```

```
                conn = get_connection()
                cur = conn.cursor()
                cur.execute("SELECT subject_name FROM subjects WHERE subject_id =
?", (data[4],))
                row = cur.fetchone()
                conn.close()
                if row:
                    self.subject_cb.addItem(row[0], data[4])
                    self.subject_cb.setCurrentIndex(self.subject_cb.count() - 1)

            self.teacher_cb.setCurrentIndex(self.teacher_cb.findData(data[5]))
            self.audience_cb.setCurrentIndex(self.audience_cb.findData(data[6]))

self.week_cb.setCurrentIndex(self.week_cb.findText(self.REVERSE_WEEK_MAP.get(dat
a[7], "Щотижня")))

        # якщо це додавання, то показуємо всі предмети за замовчуванням
        else:
            self.group_cb.setCurrentIndex(0)
            self.update_subjects_by_group()

    def load_comboboxes(self):
        conn = get_connection()
        cur = conn.cursor()

        # Дні
        cur.execute("SELECT day_id, day_name FROM days")
        for day_id, name in cur.fetchall():
            self.day_cb.addItem(name, day_id)

        # Пари
        cur.execute("SELECT lesson_id, lesson_number, start_time, end_time FROM
lessons")
        for lesson_id, number, start, end in cur.fetchall():
            self.lesson_cb.addItem(f"Пара {number} ({start}-{end})", lesson_id)

        # Групи
        self.group_cb.addItem("Усі групи", None)  # перший пункт
        cur.execute("SELECT group_id, group_name FROM groups ORDER BY group_name")
        for group_id, name in cur.fetchall():
            self.group_cb.addItem(name, group_id)

        # Предмети (поки всі, потім фільтруються)
        cur.execute("SELECT  subject_id,  subject_name  FROM  subjects  ORDER  BY
subject_name")
        for subject_id, name in cur.fetchall():
            self.subject_cb.addItem(name, subject_id)

        # Викладачі
        cur.execute("SELECT  teacher_id,  full_name  FROM  teachers  ORDER  BY
full_name")
        for teacher_id, name in cur.fetchall():
            self.teacher_cb.addItem(name, teacher_id)

        # Аудиторії
        cur.execute("SELECT  audience_id,  room_number  FROM  audiences  ORDER  BY
room_number")
        for audience_id, room in cur.fetchall():
            self.audience_cb.addItem(room, audience_id)

        conn.close()
        self.week_cb.addItems(list(self.WEEK_MAP.keys()))

    def get_data(self):
        week_text = self.week_cb.currentText()
        return {
```

```
            "day_id": self.day_cb.currentData(),
            "lesson_id": self.lesson_cb.currentData(),
            "group_id": self.group_cb.currentData(),
            "subject_id": self.subject_cb.currentData(),
            "teacher_id": self.teacher_cb.currentData(),
            "audience_id": self.audience_cb.currentData(),
            "week_type": self.WEEK_MAP.get(week_text, "both")
        }

    # ----------------------------
    # Додаткові методи
    # ----------------------------

    def get_course_from_group(self, group_name):
        """Визначає курс за назвою групи (наприклад, ІС-23 → 3 курс)."""
        import re
        match = re.search(r"(\d{2})", group_name)
        if match:
            year = int(match.group(1))
            current_year = 25  # набір ІС-25 — 1 курс
            course = current_year - year + 1
            if 1 <= course <= 4:
                return course
        return None

    def update_subjects_by_group(self):
        """Фільтрує предмети за курсом при виборі групи."""
        group_name = self.group_cb.currentText()

        # Якщо обрано "Усі групи" → показати всі предмети
        if group_name == "Усі групи":
            conn = get_connection()
            cur = conn.cursor()
            cur.execute("SELECT subject_id, subject_name FROM subjects ORDER BY
subject_name")
            subjects = cur.fetchall()
            conn.close()
        else:
            course = self.get_course_from_group(group_name)
            if not course:
                return

            semester_ranges = {
                1: (1, 2),
                2: (3, 4),
                3: (5, 6),
                4: (7, 8)
            }
            sem_start, sem_end = semester_ranges.get(course, (1, 8))

            conn = get_connection()
            cur = conn.cursor()
            cur.execute(
                "SELECT subject_id, subject_name FROM subjects WHERE semester
BETWEEN ? AND ? ORDER BY subject_name",
                (sem_start, sem_end)
            )
            subjects = cur.fetchall()
            conn.close()

        # Оновлюємо список предметів
        self.subject_cb.blockSignals(True)
        self.subject_cb.clear()
        if subjects:
            for subject_id, name in subjects:
                self.subject_cb.addItem(name, subject_id)
```

```
        else:
            self.subject_cb.addItem("Немає предметів для цього курсу", -1)
        self.subject_cb.blockSignals(False)


class ScheduleTab(QWidget):
    WEEK_MAP = {"Чисельник": "upper", "Знаменник": "lower", "Щотижня": "both"}
    REVERSE_WEEK_MAP = {v: k for k, v in WEEK_MAP.items()}

    def __init__(self):
        super().__init__()
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout()

        # --- Панель CRUD кнопок ---
        # --- Панель керування ---
        button_layout = QHBoxLayout()
        button_layout.setAlignment(Qt.AlignLeft)           # Вирівнювання кнопок
зліва
        button_layout.setSpacing(20)                      # Відстань між кнопками
— 20 px
        button_layout.setContentsMargins(0, 30, 0, 30)     # Відступи зверху і
знизу по 30 px

        self.btn_add = QPushButton("Додати")
        self.btn_edit = QPushButton("Редагувати")
        self.btn_delete = QPushButton("Видалити")

        button_layout.addWidget(self.btn_add)
        button_layout.addWidget(self.btn_edit)
        button_layout.addWidget(self.btn_delete)

        layout.addLayout(button_layout)


        # --- Панель фільтрів ---
        filter_layout = QHBoxLayout()

        self.filter_lesson_cb = QComboBox()
        self.filter_lesson_cb.addItem("Всі пари", None)
        for lesson_id, number, start, end in self.get_lessons():
            self.filter_lesson_cb.addItem(f"Пара    {number}    ({start}-{end})",
lesson_id)

        self.filter_day_cb = QComboBox()
        self.filter_day_cb.addItem("Всі дні", None)
        for day_id, name in self.get_days():
            self.filter_day_cb.addItem(name, day_id)

        self.filter_group_cb = QComboBox()
        self.filter_group_cb.addItem("Всі групи", None)
        for group_id, name in self.get_groups():
            self.filter_group_cb.addItem(name, group_id)

        self.filter_teacher_cb = QComboBox()
        self.filter_teacher_cb.addItem("Всі викладачі", None)
        for teacher_id, name in self.get_teachers():
            self.filter_teacher_cb.addItem(name, teacher_id)

        self.filter_week_cb = QComboBox()
        self.filter_week_cb.addItem("Всі тижні", None)
        self.filter_week_cb.addItems(list(self.WEEK_MAP.keys()))

        self.btn_reset_filters = QPushButton("Скинути")
```

```
        # Додаємо на панель
        filter_layout.addWidget(QLabel("Пара:"))
        filter_layout.addWidget(self.filter_lesson_cb)
        filter_layout.addWidget(QLabel("День:"))
        filter_layout.addWidget(self.filter_day_cb)
        filter_layout.addWidget(QLabel("Група:"))
        filter_layout.addWidget(self.filter_group_cb)
        filter_layout.addWidget(QLabel("Викладач:"))
        filter_layout.addWidget(self.filter_teacher_cb)
        filter_layout.addWidget(QLabel("Тиждень:"))
        filter_layout.addWidget(self.filter_week_cb)
        filter_layout.addWidget(self.btn_reset_filters)

        layout.addLayout(filter_layout)

        # --- Таблиця розкладу ---
        self.table = QTableWidget()
        self.table.setColumnCount(8)
        self.table.setHorizontalHeaderLabels([
            "ID", "День", "Пара", "Група", "Предмет", "Викладач", "Аудиторія",
"Тиждень"
        ])
        self.table.hideColumn(0)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        self.table.setSelectionMode(QTableWidget.SingleSelection)
        self.table.setColumnWidth(1, 100)
        self.table.setColumnWidth(2, 70)
        self.table.setColumnWidth(3, 90)
        self.table.setColumnWidth(4, 250)
        self.table.setColumnWidth(5, 200)
        self.table.setColumnWidth(6, 100)
        self.table.setColumnWidth(7, 80)
        layout.addWidget(QLabel("Розклад занять:"))
        layout.addWidget(self.table)
        self.setLayout(layout)

        # --- Підключення кнопок ---
        self.btn_add.clicked.connect(self.add_schedule)
        self.btn_edit.clicked.connect(self.edit_schedule)
        self.btn_delete.clicked.connect(self.delete_schedule)
        self.btn_reset_filters.clicked.connect(self.reset_filter)
        self.filter_day_cb.currentIndexChanged.connect(self.apply_filter)
        self.filter_lesson_cb.currentIndexChanged.connect(self.apply_filter)
        self.filter_group_cb.currentIndexChanged.connect(self.apply_filter)
        self.filter_teacher_cb.currentIndexChanged.connect(self.apply_filter)
        self.filter_week_cb.currentIndexChanged.connect(self.apply_filter)

        self.load_schedule()


    # --- Методи для отримання даних ---
    def get_lessons(self):
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT lesson_id, lesson_number, start_time, end_time FROM
lessons ORDER BY lesson_number")
        rows = cur.fetchall()
        conn.close()
        return rows

    def get_days(self):
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT day_id, day_name FROM days ORDER BY day_id")
        rows = cur.fetchall()
```

```
        conn.close()
        return rows

    def get_groups(self):
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT group_id, group_name FROM groups ORDER BY group_name")
        rows = cur.fetchall()
        conn.close()
        return rows

    def get_teachers(self):
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT  teacher_id,  full_name  FROM  teachers  ORDER  BY
full_name")
        rows = cur.fetchall()
        conn.close()
        return rows



    # --- Завантаження розкладу ---
    def load_schedule(self, filtered_rows=None):
        rows   =   filtered_rows   if   filtered_rows   is   not   None   else
self.fetch_schedule()
        self.table.setRowCount(len(rows))

        for i, row in enumerate(rows):
            (schedule_id, day_id, day_name, lesson_id, lesson_number, start_time,
end_time,
             group_id,   group_name,   subject_id,   subject_name,   teacher_id,
teacher_name,
             audience_id, room_number, week_type) = row

            # ID
            id_item = QTableWidgetItem(str(schedule_id))
            id_item.setData(Qt.UserRole, schedule_id)
            self.table.setItem(i, 0, id_item)

            # День
            day_item = QTableWidgetItem(day_name)
            day_item.setData(Qt.UserRole, day_id)
            self.table.setItem(i, 1, day_item)

            # Пара + підказка
            lesson_item = QTableWidgetItem(f"Пара {lesson_number}")
            lesson_item.setToolTip(f"{start_time} - {end_time}")
            lesson_item.setData(Qt.UserRole, lesson_id)
            self.table.setItem(i, 2, lesson_item)

            # Група
            group_item = QTableWidgetItem(group_name)
            group_item.setData(Qt.UserRole, group_id)
            self.table.setItem(i, 3, group_item)

            # Предмет
            subject_item = QTableWidgetItem(subject_name)
            subject_item.setData(Qt.UserRole, subject_id)
            self.table.setItem(i, 4, subject_item)

            # Викладач
            teacher_item = QTableWidgetItem(teacher_name)
            teacher_item.setData(Qt.UserRole, teacher_id)
            self.table.setItem(i, 5, teacher_item)
```

```
            # Аудиторія
            aud_item = QTableWidgetItem(room_number)
            aud_item.setData(Qt.UserRole, audience_id)
            self.table.setItem(i, 6, aud_item)

            # Тиждень
            week_item   =   QTableWidgetItem(self.REVERSE_WEEK_MAP.get(week_type,
"Щотижня"))
            self.table.setItem(i, 7, week_item)

    # --- Отримати розклад з БД ---
    def fetch_schedule(self):
        conn = get_connection()
        cur = conn.cursor()
        query = """
            SELECT    s.schedule_id,    d.day_id,    d.day_name,    l.lesson_id,
l.lesson_number, l.start_time, l.end_time,
                   g.group_id, g.group_name, sub.subject_id, sub.subject_name,
                   t.teacher_id,   t.full_name,   a.audience_id,   a.room_number,
s.week_type
            FROM schedule s
            JOIN days d ON s.day_id = d.day_id
            JOIN lessons l ON s.lesson_id = l.lesson_id
            JOIN groups g ON s.group_id = g.group_id
            JOIN subjects sub ON s.subject_id = sub.subject_id
            JOIN teachers t ON s.teacher_id = t.teacher_id
            JOIN audiences a ON s.audience_id = a.audience_id
            ORDER BY d.day_id, l.lesson_number
        """
        cur.execute(query)
        rows = cur.fetchall()
        conn.close()
        return rows

    # --- Додати ---
    def add_schedule(self):
        dialog = ScheduleDialog(self)
        if dialog.exec_():
            data = dialog.get_data()
            conn = get_connection()
            cur = conn.cursor()
            cur.execute("""
                INSERT INTO schedule (day_id, lesson_id, group_id, subject_id,
teacher_id, audience_id, week_type)
                VALUES (?, ?, ?, ?, ?, ?, ?)
            """,     (data["day_id"],     data["lesson_id"],     data["group_id"],
data["subject_id"],
                  data["teacher_id"], data["audience_id"], data["week_type"]))
            conn.commit()
            conn.close()
            self.load_schedule()

    # --- Редагувати ---
    def edit_schedule(self):
        selected_row = self.table.currentRow()
        if selected_row < 0:
            QMessageBox.warning(self,    "Помилка",    "Виберіть    рядок    для
редагування!")
            return

        data = [
            int(self.table.item(selected_row, 0).data(Qt.UserRole)),
            int(self.table.item(selected_row, 1).data(Qt.UserRole)),
            int(self.table.item(selected_row, 2).data(Qt.UserRole)),
            int(self.table.item(selected_row, 3).data(Qt.UserRole)),
            int(self.table.item(selected_row, 4).data(Qt.UserRole)),
```

```
            int(self.table.item(selected_row, 5).data(Qt.UserRole)),
            int(self.table.item(selected_row, 6).data(Qt.UserRole)),
            self.WEEK_MAP.get(self.table.item(selected_row, 7).text(), "both")
        ]

        dialog = ScheduleDialog(self, data)
        if dialog.exec_():
            new_data = dialog.get_data()
            conn = get_connection()
            cur = conn.cursor()
            cur.execute("""
                UPDATE schedule
                SET    day_id=?,    lesson_id=?,    group_id=?,    subject_id=?,
teacher_id=?, audience_id=?, week_type=?
                WHERE schedule_id=?
            """,           (new_data["day_id"],           new_data["lesson_id"],
new_data["group_id"], new_data["subject_id"],
                  new_data["teacher_id"],               new_data["audience_id"],
new_data["week_type"], data[0]))
            conn.commit()
            conn.close()
            self.load_schedule()

    # --- Видалити ---
    def delete_schedule(self):
        selected_row = self.table.currentRow()
        if selected_row < 0:
            QMessageBox.warning(self, "Помилка", "Виберіть рядок для видалення!")
            return

        schedule_id = int(self.table.item(selected_row, 0).data(Qt.UserRole))
        reply = QMessageBox.question(self, "Підтвердження", "Видалити цей запис
розкладу?",
                                     QMessageBox.Yes | QMessageBox.No)
        if reply == QMessageBox.Yes:
            conn = get_connection()
            cur = conn.cursor()
            cur.execute("DELETE    FROM    schedule    WHERE    schedule_id=?",
(schedule_id,))
            conn.commit()
            conn.close()
            self.load_schedule()

    # --- Фільтрувати ---
    def apply_filter(self):
        day_id = self.filter_day_cb.currentData()
        lesson_id = self.filter_lesson_cb.currentData()
        group_id = self.filter_group_cb.currentData()
        teacher_id = self.filter_teacher_cb.currentData()
        week_text = self.filter_week_cb.currentText()
        week_value = self.WEEK_MAP.get(week_text, None)

        query = """
            SELECT    s.schedule_id,    d.day_id,    d.day_name,    l.lesson_id,
l.lesson_number, l.start_time, l.end_time,
                   g.group_id, g.group_name, sub.subject_id, sub.subject_name,
                   t.teacher_id,   t.full_name,   a.audience_id,   a.room_number,
s.week_type
            FROM schedule s
            JOIN days d ON s.day_id = d.day_id
            JOIN lessons l ON s.lesson_id = l.lesson_id
            JOIN groups g ON s.group_id = g.group_id
            JOIN subjects sub ON s.subject_id = sub.subject_id
            JOIN teachers t ON s.teacher_id = t.teacher_id
            JOIN audiences a ON s.audience_id = a.audience_id
            WHERE 1=1
```

```
        """
        params = []
        if day_id:
            query += " AND s.day_id=?"
            params.append(day_id)
        if lesson_id:
            query += " AND s.lesson_id=?"
            params.append(lesson_id)
        if group_id:
            query += " AND s.group_id=?"
            params.append(group_id)
        if teacher_id:
            query += " AND s.teacher_id=?"
            params.append(teacher_id)
        if week_value:
            query += " AND s.week_type=?"
            params.append(week_value)

        query += " ORDER BY d.day_id, l.lesson_number"

        conn = get_connection()
        cur = conn.cursor()
        cur.execute(query, params)
        rows = cur.fetchall()
        conn.close()

        self.load_schedule(filtered_rows=rows)

    # --- Скинути фільтри ---
    def reset_filter(self):
        self.filter_day_cb.setCurrentIndex(0)
        self.filter_lesson_cb.setCurrentIndex(0)
        self.filter_group_cb.setCurrentIndex(0)
        self.filter_teacher_cb.setCurrentIndex(0)
        self.filter_week_cb.setCurrentIndex(0)
        self.load_schedule()
```